

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – Coding Challenge

COURSE NAME: Data Structures

COURSE CODE:CSA0302

COURSE OUTCOME COVERED

CO: Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data (BL3)

Assessment Tool Weightage: 35%

QUESTIONS: EACH QUESTION CARRIES: 10 MARKS DURATION : 1 HOUR

1. Implement a hash table for integer keys and string values using separate chaining with singly linked lists.
Requirements:
 - 1) Hash function: $\text{hash}(\text{key}) = \text{key} \% \text{table_size}$ (but implement your own polynomial rolling hash for string keys).
 - 2) Operations: insert, search, delete, display.
 - 3) Handle collisions using linked lists (dynamically allocated nodes).
 - 4) Implement rehash() when load factor > 1.5 (double table size and reinsert all elements).
 - 5) Free all memory properly (no leaks).

2. Implement a min-heap using a dynamic array, then use it to implement heap sort for an integer array.
Requirements:
 - 1) heapify() — build heap from unsorted array in $O(n)$.
 - 2) insert() — add element, maintain heap property.
 - 3) extract_min() — remove and return minimum.
 - 4) heap_sort() — sort array in-place using heap (ascending order).
 - 5) Functions for parent(), left_child(), right_child().
 - 6) Implement sift_up() and sift_down().

3. Implement LSD (Least Significant Digit) radix sort for integers and MSD (Most Significant Digit) radix sort for strings.
Requirements (Integers):
 - 1) Base = 10 (decimal) or base = 256 (byte-level).

- 2) Stable counting sort per digit.
- 3) Handle negative numbers (bias by adding min value).
Requirements (Strings):
 - 1) MSD radix sort with 256 possible characters (extended ASCII).
 - 2) Use insertion sort for small buckets (size < 50).
 - 3) Handle variable-length strings (null-terminated).

4. Simulate sorting of a file too large to fit into memory.
Tasks:
 - 1) Divide data into chunks
 - 2) Sort each chunk (use Merge Sort/Heap Sort)
 - 3) Merge all chunks using k-way merging (Min Heap)
 - 4) Count number of disk reads/writes (simulate)

5. Implement a hash table that automatically resizes when load factor exceeds a threshold (e.g., 0.75).
Tasks:
 - 1) Double table size
 - 2) Rehash all elements
 - 3) Maintain performance

6. Implement merge sort on a singly linked list (instead of array).
Requirements:
 - 1) Node structure: int data and struct Node* next.
 - 2) Functions: split_list() (find middle using slow/fast Pointers), merge_sorted().
 - 3) Recursive merge sort on linked list.
 - 4) Print original and sorted list.

7. Implement a hash table using double hashing (open addressing) with dynamic resizing.
Requirements:
 - 1) Hash functions:

$$h1(key) = key \% size$$

$$h2(key) = 1 + (key \% (size - 1))$$
 Probe sequence: $h(k, i) = (h1(k) + i * h2(k)) \% size$
 - 2) Load factor threshold: 0.7 (resize to next prime number $\times$ 2).
 - 3) Support insert(), search(), delete() (use tombstone marker).
 - 4) After resizing, rehash all elements with new size.
 - 5) Find next prime number for new table size.

8. Implement a generic priority queue (heap) that can store any data type using void* and function pointers.
Requirements:

- 1) Store void* elements with size information.
 - 2) Accept comparator function: `int (*compare)(const void*, const void*)` (returns negative if $a < b$).
 - 3) Support `push()`, `pop()`, `peek()`, `size()`.
 - 4) Dynamically grow array when full (double capacity).
 - 5) Generic `heapify_up()` and `heapify_down()` using comparator.
- 9.** Simulate external sorting for a large file that doesn't fit in memory.
Requirements:
- 1) Generate a large array of random integers (e.g., 100,000).
 - 2) Split into K chunks that fit in memory (e.g., 10 chunks of 10,000 each).
 - 3) Sort each chunk using quick sort and write to temporary files.
 - 4) Merge all sorted chunks using a min-heap (K-way merge):
 - a. Read first element from each chunk into heap.
 - b. Extract min, write to output, read next from same chunk.
 - 5) Clean up temporary files.
- 10.** Implement parallel quick sort using a fixed thread pool (no creating unlimited threads).
Requirements:
- 1) Create T threads at start (e.g., $T = 4$).
 - 2) Use a task queue (simple array or linked list) to store sorting tasks: (array, left, right).
 - 3) Main thread pushes initial task, worker threads pick tasks and process.
 - 4) If partition size < 1000 , sort sequentially (insertion sort).
 - 5) If partition size ≥ 1000 , split into two tasks and push to queue.
 - 6) Use mutex for task queue access and condition variables for waiting.
- 11.** Implement a hybrid sorting algorithm combining merge sort and insertion sort (similar to Timsort's basic idea).
Requirements:
- 1) Scan array to identify natural runs (already sorted sequences).
 - 2) If run length $< \text{MIN_RUN}$ (e.g., 32), extend run and sort using insertion sort.
 - 3) Push runs onto a stack (store start index, length).
 - 4) Merge runs when top runs have lengths satisfying:
 $\text{len}(\text{top}-2) \leq \text{len}(\text{top}-1) + \text{len}(\text{top})$ and $\text{len}(\text{top}-1) \leq \text{len}(\text{top})$ (similar to Timsort's merge invariants).
 - 5) Use temporary array for merging.
- 12.** Implement Cuckoo hashing with two hash tables and two hash functions.
Requirements:
- 1) Two arrays of equal size (size = next prime $> 2 \times$ expected elements).
 - 2) Hash functions: $h1(\text{key}) = \text{key} \% \text{size}$, $h2(\text{key}) = (\text{key} / \text{size}) \% \text{size}$.

- 3) Insert: Try h_1 , if occupied, kick out existing key and reinsert it using alternate hash.
 - 4) Loop detection: If kicks exceed threshold (e.g., 500), rehash with larger table.
 - 5) Support `insert()`, `search()`, `delete()`.
- 13.** Implement a Leftist Heap — a mergeable priority queue with $O(\log n)$ merge operation.
Requirements:
- 1) Node structure: value, dist (null path length), left, right.
 - 2) Operations: `merge()`, `insert()`, `extract_min()`, `build_heap()`.
 - 3) Maintain leftist property: $\text{dist}(\text{left}) \geq \text{dist}(\text{right})$ for every node.
 - 4) $\text{dist}(\text{NULL}) = 0$, $\text{dist}(\text{node}) = \text{dist}(\text{right}) + 1$.
- 14.** Implement a stable merge sort that works with any data type using function pointers and preserves original order for equal elements.
Requirements:
- 1) Generic function: `void stable_sort(void *base, size_t n, size_t elem_size, int (*compare)(const void*, const void*))`.
 - 2) Use temporary array for merging.
 - 3) Maintain stability: If `compare(a, b) == 0`, `a` comes before `b` in output.
 - 4) Test with array of structures (e.g., `{id, name}`) sorting by name while preserving id order.
- 15.** Implement Bitonic sort — a parallel sorting network suitable for GPU/OpenMP.
Requirements:
- 1) Sort arrays of size n (must be power of two).
 - 2) Functions: `bitonic_sort()`, `bitonic_merge()`, `compare_and_swap()`.
 - 3) Implement both recursive and iterative versions.
 - 4) For parallel version (bonus), use OpenMP `#pragma omp parallel` for.
- 16.** Implement counting sort for integers in range $[0, K]$ and extend to radix sort for large ranges.
Requirements:
- 1) Counting sort: $O(n + K)$ time, $O(K)$ space.
 - 2) Radix sort: Sort by each digit (base 10 or base 256) using stable counting sort.
 - 3) Handle negative numbers by shifting (add min value).
 - 4) Compare performance with quick sort for different K values.
- 17.** Implement a priority queue supporting `decrease_key()` operation (useful for Dijkstra's algorithm).
Requirements:
- 1) Store (key, value) pairs.

- 2) Maintain a map (array or hash table) from value to its index in heap.
 - 3) Operations: insert(), extract_min(), decrease_key(value, new_key).
 - 4) After decrease_key, sift up the element.
 - 5) Update index map whenever elements move.
- 18.** Implement replacement selection to generate longer initial runs for external sorting.
Requirements:
- 1) Read input from a large file (simulated with array).
 - 2) Use a min-heap of size M (e.g., 1000).
 - 3) Algorithm: Read M elements into heap, output min, read next element if greater than last output, else hold for next run.
 - 4) Generate multiple runs, then merge them using K-way merge.
- 19.** Implement parallel merge sort using OpenMP for shared-memory parallelism.
Requirements:
- 1) Recursive merge sort with OpenMP #pragma omp task.
 - 2) Set cutoff: if size < 10000, sort sequentially (insertion sort).
 - 3) Use #pragma omp parallel and #pragma omp single to start.
 - 4) #pragma omp taskwait to synchronize.
 - 5) Measure speedup on 1, 2, 4, 8 threads.
- 20.** Implement Robin Hood hashing — an open addressing method that balances probe lengths.
Requirements:
- 1) Each entry stores (key, value, distance_from_desired).
 - 2) On insert: if new element has smaller probe distance than existing, swap and continue.
 - 3) On delete: use backward shift deletion (shift elements backward to fill holes).
 - 4) Maintain low variance in probe lengths.
- 21.** Implement a hash table using linear probing where the probe stride starts at 1 but doubles after each collision (1, 2, 4, 8, ...). Wrap around using modulo. Support insert, search, and delete. Compare its average probe count against standard linear probing for load factors 0.5, 0.7, and 0.9.
- 22.** Implement a hash table with two independent hash functions h1 and h2. For each insertion, compute both hash values and insert the key into the bucket (chain) that currently has fewer elements. If equal, choose h1. Support insert, search, and delete. Compare the maximum chain length against standard single-hash separate chaining.
- 23.** Design a data structure that maintains a fixed-size array of key-value pairs. Support insert (add at next free slot, then bubble up by key priority) and extract-min (remove root, replace with last element, bubble down). Also support hash-

based lookup using an index map. This is essentially a heap with $O(1)$ lookup by key.

24. Implement a queue using two heaps: an insertion heap (max-heap) and a deletion heap (min-heap). On enqueue, push to insertion heap. On dequeue, if deletion heap is empty, pop all elements from insertion heap and push into deletion heap, then pop from deletion heap. Analyze amortized time complexity.
25. Implement odd-even transposition sort for an array of size N . This is a parallel sorting network suitable for SIMD or multi-threading. Use pthreads to compare and swap all odd-even pairs in parallel, then all even-odd pairs, repeating until sorted. Compare its parallel speedup against sequential bubble sort.
26. Implement a hash table (linear probing) where delete only marks a slot as "deleted" without shifting. Every 1000 operations, trigger a cleanup that rehashes all live entries into a new table to remove tombstone buildup. Measure the performance impact of lazy deletion vs. immediate cleanup.
27. Implement an in-place radix sort for single-precision floating-point numbers (float). Treat the binary representation as a 32-bit integer, but handle sign bits correctly (negative numbers should come before positive). Use binary LSD radix sort. Verify correctness for special values (0, -0, NaN, infinity).
28. Given a large array, implement parallel heapify using pthreads. Divide the array into P segments, each handled by a thread that heapifies its segment. Then merge the P heaps into a single heap using repeated merging. Compare the time against single-threaded Floyd heapify for 100 million integers.
29. Implement a hash table where each key has an associated timestamp. Support insert (key, value, ttl_seconds). A background thread or lazy cleanup on lookup removes entries older than their TTL. Use separate chaining. Simulate real-time sensor data insertion and expiry.
30. Implement merge sort that merges two sorted halves without using $O(n)$ extra space. Use block merging (also known as in-place merge). The algorithm works by rearranging blocks of elements. Analyze its time complexity and compare memory usage against standard merge sort.
31. Implement a priority queue of fixed size K . When full, insert evicts the smallest priority element (min-heap). Additionally, support an LRU (least recently used) eviction policy for elements with equal priority. Each element stores key, priority, and last-access timestamp. Show how heap + timestamp array achieves this.
32. Implement cuckoo hashing with 3 hash tables and 3 hash functions. During insertion, if a cycle is detected, instead of rehashing immediately, try to move the offending element to another of its 3 tables. Support dynamic insertions and deletions. Measure the maximum load factor achievable before rehash is required.
33. Implement counting sort for integers in range $[0, K-1]$ using P threads. Each thread processes a disjoint segment of the input array and builds a local count array. Then combine local counts (reduction step) and compute final positions.

Finally, place elements in output array in parallel. Analyze speedup and thread synchronization overhead.

34. Implement heap sort using an iterative (non-recursive) sift-down function that uses a loop instead of function calls. Compare performance against recursive heap sort. Also, use sentinel values to reduce bounds checking. Measure the difference in CPU cycles for arrays of size 10 million.
35. Write a program that benchmarks at least 5 different hash functions for integer keys and string keys. Measure: (a) distribution quality (variance of bucket sizes), (b) avalanche effect (bit flips), (c) speed (hashes per second). Present a ranked recommendation for a general-purpose hash table.
36. Write CUDA-style pseudocode (or simulation in C with threads) for Batcher's odd-even merge sort, which is a parallel sorting network for power-of-two sizes. It consists of $\log^2 n$ stages. Show the kernel structure, shared memory usage, and compare its theoretical steps against bitonic sort. Explain why it is less common in practice.
37. Implement a hash table where the main table is an array of fixed size. Each bucket contains a pointer to a binary search tree (BST) instead of a linked list. For small buckets, you can keep the BST simple; for large buckets, the BST improves search to $O(\log m)$ per bucket where m is bucket size. Compare performance against separate chaining.
38. Implement quick sort with four different pivot selection strategies: (a) first element, (b) middle element, (c) median-of-three, (d) random pivot. For each, measure the number of comparisons and swaps on sorted, reverse-sorted, random, and partially sorted arrays of size 1 million. Present a table of results and recommend the best general-purpose strategy.
39. Combine replacement selection with multi-pass K-way merge for files larger than disk space. Implement a 2-pass external sort: first generate runs using replacement selection, then perform a K-way merge where K is limited by available file handles. If runs exceed K, perform multiple merge passes. Write to temporary files and clean up.
40. Simulate MapReduce sorting where each mapper sorts its local data first (combiner optimization) before sending to reducers. Use a fixed number of mappers (M) and reducers (R). Each mapper reads a chunk, sorts it using quick sort, and writes to R output files (one per reducer). Each reducer reads all mapper outputs and merges them using a heap. Compare with and without combiner optimization.



SIMATS Online Programming Lab
DS PROGRAMMING


Bedudoori Siva Parvathi
192525194RC

Coding Challenge

QUESTIONS

Q1
Separate Chaining Hash Table
10 marks

Q2
Min Heap Operations
10 marks

Q3
LSD Radix Sort with Negatives
10 marks

3/3 answered

Q3 LSD Radix Sort with Negatives

10 marks

Read N followed by N integers and sort them in ascending order using stable LSD radix sort. Negative values must be handled correctly. Print the sorted integers space-separated.

Your Code

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void radixSort(int a[], int n) {
5     int *out = malloc(n * sizeof(int));
6     int *pos = malloc(n * sizeof(int));
7     int *neg = malloc(n * sizeof(int));
8     int np = 0, nn = 0;
9
10    for (int i = 0; i < n; i++)
11        if (a[i] < 0) neg[nn++] = -a[i];
12        else pos[np++] = a[i];
13
14    // Stable LSD radix sort
15    void sort(int x[], int m) {
16        if (m <= 1) return;
17
18        int max = x[0];
19        for (int i = 1; i < m; i++)
20            if (x[i] > max) max = x[i];
21
22        for (long long exp = 1; max / exp > 0; exp *= 10) {
23            int c[10] = {0};
24
25            for (int i = 0; i < m; i++)
26                c[(x[i] / exp) % 10]++;
27
28            for (int i = 1; i < 10; i++)
29                c[i] += c[i - 1];
30
31            for (int i = m - 1; i >= 0; i--)
32                out[--c[(x[i] / exp) % 10]] = x[i];
33
34            for (int i = 0; i < m; i++)
35                x[i] = out[i];
36        }
37    }
38
39    sort(pos, np);
40    sort(neg, nn);
41    for (int i = 0; i < np; i++)
42        a[i] = pos[i];
43    for (int i = np; i < np + nn; i++)
44        a[i] = neg[i - np];
45    free(pos); free(neg);
46 }
47
48 int main() {
49     int n;
50     scanf("%d", &n);
51     int a[n];
52     for (int i = 0; i < n; i++)
53         scanf("%d", &a[i]);
54     radixSort(a, n);
55     for (int i = 0; i < n; i++)
56         printf("%d ", a[i]);
57     printf("\n");
58     return 0;
59 }

```

Input

```

8
170 -45 75 -90 802 24 2 -66

```

Output

```

-90 -66 -45 2 24 75 170 802

```

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

10.00 / 10 marks



SIMATS Online Programming Lab
DS PROGRAMMING


Bedudoori Siva Parvathi
192525194RC

Coding Challenge

QUESTIONS

Q1
Separate Chaining Hash Table
10 marks

Q2
Min Heap Operations
10 marks

Q3
LSD Radix Sort with Negatives
10 marks

3/3 answered

Q1 Separate Chaining Hash Table

10 marks

Implement an integer-key/string-value hash table using separate chaining. Input: table size S and Q operations. Operations are I key value, S key, D key, and P. For S print the value or NOT FOUND. For P print all stored keyvalue pairs sorted by key. Resize to double size when load factor exceeds 1.5.

Your Code

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 typedef struct N {
5     int k;
6     char v[30];
7     struct N *next;
8 } N;
9
10 N **t; int S, n=0;
11 int h(int k){ return (k%S+S)%S; }
12 void resize(){
13     N **oldt; int oldS=S; S*=2;
14     t=malloc(S*sizeof(N));
15     for(int i=0;i<oldS;i++)
16         while(oldt[i]){
17             N *p=oldt[i]; oldt[i]=p->next;
18             int x=h(p->k);
19             p->next=t[x]; t[x]=p;
20         }
21     free(oldt);
22 }
23 void insert(int k,char *v){
24     int x=h(k); N *p=t[x];
25     while(p){
26         if(p->k==k){ strcpy(p->v,v); return; }
27         p=p->next;
28     }
29     p=malloc(sizeof(N));
30     p->k=k; strcpy(p->v,v);
31     p->next=t[x]; t[x]=p; n++;
32
33     if((float)n/S>1.5) resize();
34 }
35 void search(int k){
36     for(N *p=t[h(k)];p;p=p->next)
37         if(p->k==k){ printf("%s\n",p->v); return; }
38     printf("NOT FOUND\n");
39 }

```

Input

```

5 7
I 10 Apple
I 20 Mango
I 5 Orange
...

```

Output

```

Mango
NOT FOUND
5:Orange 20:Mango

```

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

10.00 / 10 marks

SIMATS Online Programming Lab
DS PROGRAMMING

Coding Challenge
← Back

QUESTIONS

Q1
Separate Chaining Hash Table
10 marks

Q2
Min Heap Operations
10 marks

Q3
LSD Radix Sort with Negatives
10 marks

3/3 answered

Q2 Min Heap Operations
10 marks

Implement a dynamic min-heap. Input: Q operations I x, E (extract minimum), or P (peek). Print the result of every E and P; print EMPTY when the heap is empty.

Your Code

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 int *h, n = 0, cap = 2;
4 void insert(int x) {
5     if (n == cap) {
6         cap *= 2;
7         h = realloc(h, cap * sizeof(int));
8     }
9     int i = n++;
10    while (i > 0 && h[(i-1)/2] > x) {
11        h[i] = h[(i-1)/2];
12        i = (i-1)/2;
13    }
14    h[i] = x;
15 }
16 void extractMin() {
17     if (n == 0) { printf("EMPTY\n"); return; }
18     int x = h[0], last = h[--n];
19     printf("%d\n", x);
20     int i = 0, c;
21     while ((c = 2*i+1) < n) {
22         if (c+1 < n && h[c+1] < h[c]) c++;
23         if (h[c] >= last) break;
24         h[i] = h[c];
25         i = c;
26     }
27     if (n) h[i] = last;
28 }
29 void peek() {
30     if (n == 0) printf("EMPTY\n");
31     else printf("%d\n", h[0]);
32 }
33 int main() {
34     int Q, x;
35     char op;
36     h = malloc(cap * sizeof(int));
37     scanf("%d", &Q);
38     while (Q--) {

```

Input

```

8
I 30
I 10
I 20

```

Output

```

10
10
20
20
30

```

Visible Test Cases

Test Case 1 ✓

Test Case 2 ✓

Test Case 3 ✓

10.00 / 10 marks

Code of Conduct:

I B. Siva Parvathi(192525194) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B.Siva Parvathi

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developin g	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases